NEXTBOOK · LARAVEL 12 · POSTGRESQL 16

The database, domain by domain

Sixty-one application tables, drawn as eight working diagrams. Every money event in the system converges on the same two tables — `transactions` and `transaction_lines` — and most of the rest of the schema is either what feeds them or what reads them back.

APPLICATION TABLES	PRIMARY KEYS	TENANCY COLUMN	POSTING MODEL	STOCK COSTING
61 plus Laravel's cache, jobs and session tables	ULID `char(26)`, never auto-increment	branch_id on 46 of 61 tables	Double entry Σ debit = Σ credit per transaction	FIFO layers in `stock_movements`

READ THIS FIRST

Four conventions that hold everywhere

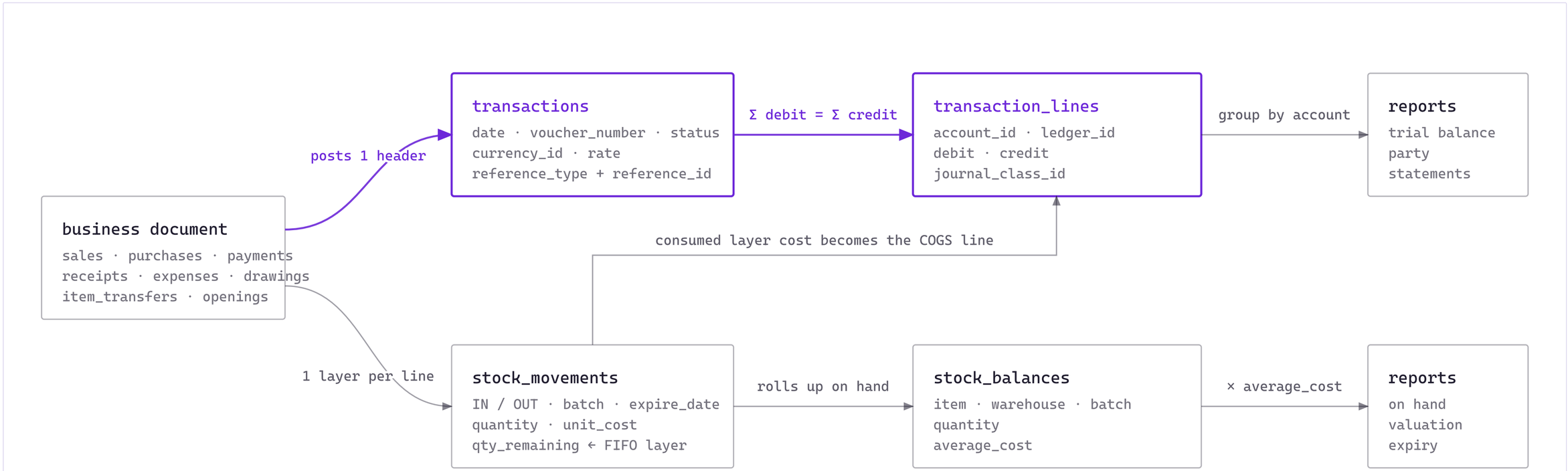
`id char(26)`	Every table has a ULID primary key and every foreign key is a `ulid` column. Sort order is creation order, so **no table needs a sequence**.
`branch_id`	The tenancy key in the data is the **branch**, not the company. Only `users` and `invoice_formats` carry `company_id`; everything else reaches the company through the user's branch. Scoping is applied by the `BranchSpecific` trait and controllers — **there is no global query scope and no database-level guarantee**.
`created_by / updated_by / deleted_by`	Audit columns FK to `users` on nearly every table, and most tables are soft-deleted. Uniqueness is declared as `(branch_id, name, deleted_at)` so a deleted record never blocks a new one with the same name.
`amount + rate`	Money is stored at `decimal(19,4)` or `decimal(18,4)`. Foreign currency is not stored per line — the **rate lives once on the transaction header** (`transactions.currency_id`, `transactions.rate`), and daily rates are journalled in `currency_rate_updates`.

Diagram notation: `||--o{` one to many `|o--o{` optional to many `||--|{` one to one-or-more `}o--o{` many to many

THE MECHANISM · 1 FIGURE

How a business document becomes a balance

Nothing in the app writes a balance directly. A sale, purchase, payment or expense is saved as its own document, and the posting layer then fans it out along two independent paths: the general ledger, and — only for documents that move goods — the FIFO stock layers.



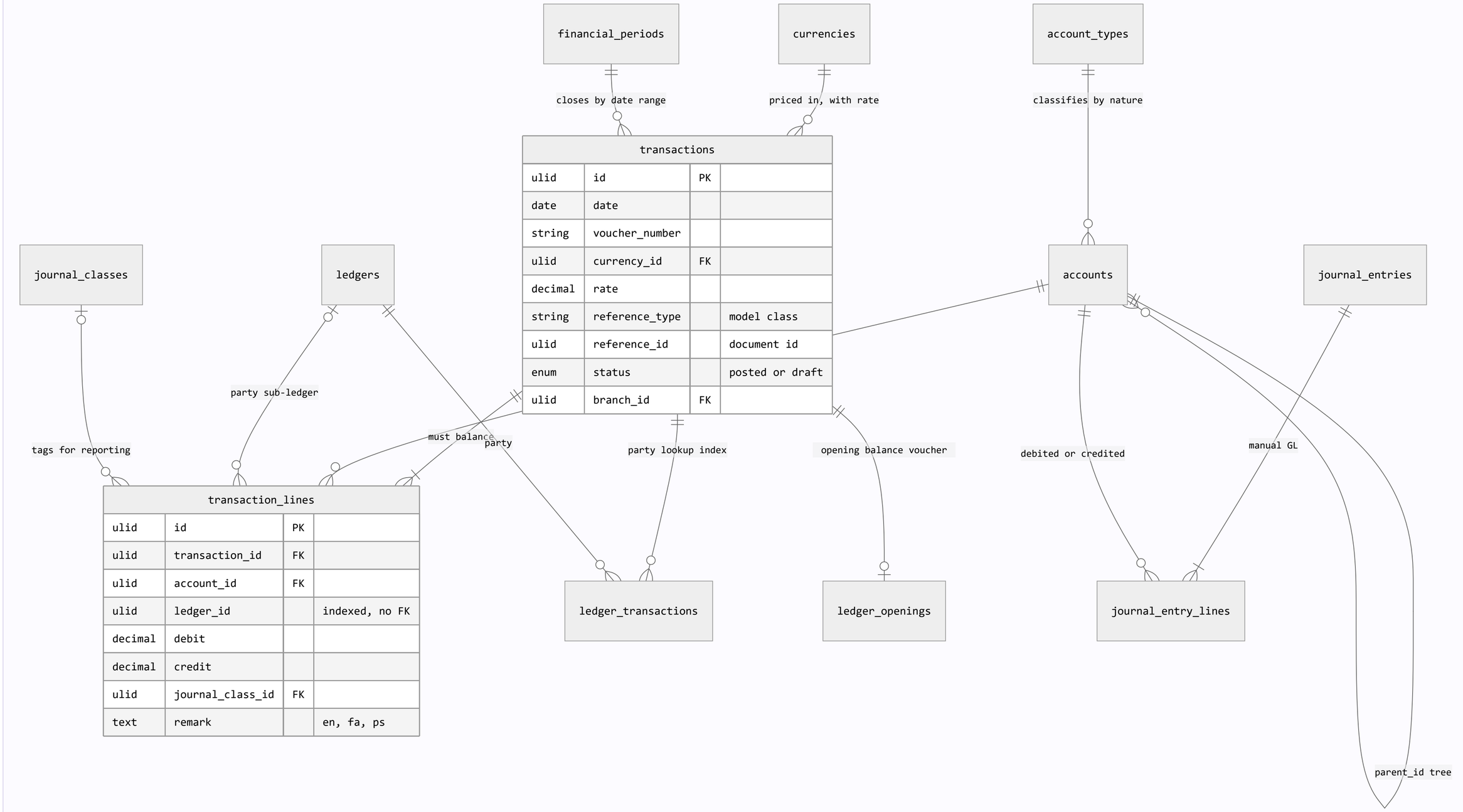
The two paths are separate writes, not one. Quantities live only in `stock_movements` / `stock_balances`; money lives only in `transaction_lines`. They meet at one point — the cost of the FIFO layers a sale consumes is what the cost-of-goods line is worth — which is why a stock write that succeeds while its posting fails leaves the books out of step with the shelf.

11 TABLES · ACCOUNTING ENGINE

Chart of accounts and the posting engine

`accounts` is a tree classified by `account_types.nature`. Everything financial lands in `transaction_lines`, either through the automatic path (`transactions`, written by the app) or the manual path (`journal_entries`, typed by an accountant).

PLATE 1 — GENERAL LEDGER



`transaction_lines` is the one table every financial report reads. It deliberately carries no `branch_id` and no `created_by` — it inherits both from its header row, which means every query against it must join `transactions` to stay inside the tenant.

`ledger_transactions`	Not a real relation of its own — a **lookup table** pairing a transaction with a party so customer and supplier statements don't have to scan every line.
`accounts`	`number`, `account_type_id`, `parent_id`, `is_main`, `is_active` — a numbered tree whose `account_types.nature` decides which side of the trial balance a balance lands on.
`ledger_openings`	Polymorphic (`ledgerable_type` / `ledgerable_id`) so an opening balance can hang off a customer, a supplier or an account, while `transaction_id` points at the ordinary transaction that carries the amounts.
`financial_periods`	Joined to nothing. A period is a date range plus a status; closing is enforced in application code by comparing `transactions.date`, so **a direct SQL insert can still write into a closed period**.
`journal_classes`	A free reporting dimension on each line — the schema's only cost-centre-like tag.

1 POLYMORPHIC COLUMN · 13 SOURCES

What is allowed to post to the ledger

`transactions.reference_type` stores a model class name and `reference_id` the document's ULID. There is no foreign key on that pair, so this diagram is the only place the set of legal sources is written down — it is a convention held by the posting services, not by the database.

REFERENCE_TYPE	DOCUMENT TABLE	WHAT THE POSTING SAYS
`Sale`	`sales`	Debit the customer or cash, credit income; a second pair moves stock value to cost of goods.
`Purchase`	`purchases`	Debit inventory, credit the supplier or the `bank_account_id` named on the purchase.
`Receipt`	`receipts`	Debit a cash or bank account, credit the customer.
`Payment`	`payments`	Debit the supplier, credit a cash or bank account.
`Expense`	`expenses`	One debit per `expense_details` row, credited to the paying account.
`AccountTransfer`	`account_transfers`	Two lines between own accounts. The only document that stores a `transaction_id` instead of being pointed at.
`JournalEntry`	`journal_entries`	Free-form manual entry, mirrored from `journal_entry_lines`.
`Drawing`	`drawings`	Debit the owner's drawing account, credit cash.
`Owner`	`owners`	Opening capital contribution against `capital_account_id`.
`ItemTransfer`	`item_transfers`	Value moved between warehouses, plus any `transfer_cost`.
`Item`	`items`	Opening stock value, linked through `item_opening_transactions`.
`Ledger`	`ledgers`	Opening balance for a customer or supplier, via `ledger_openings`.
`Account`	`accounts`	Opening balance for a GL account, via `ledger_openings` polymorphism.

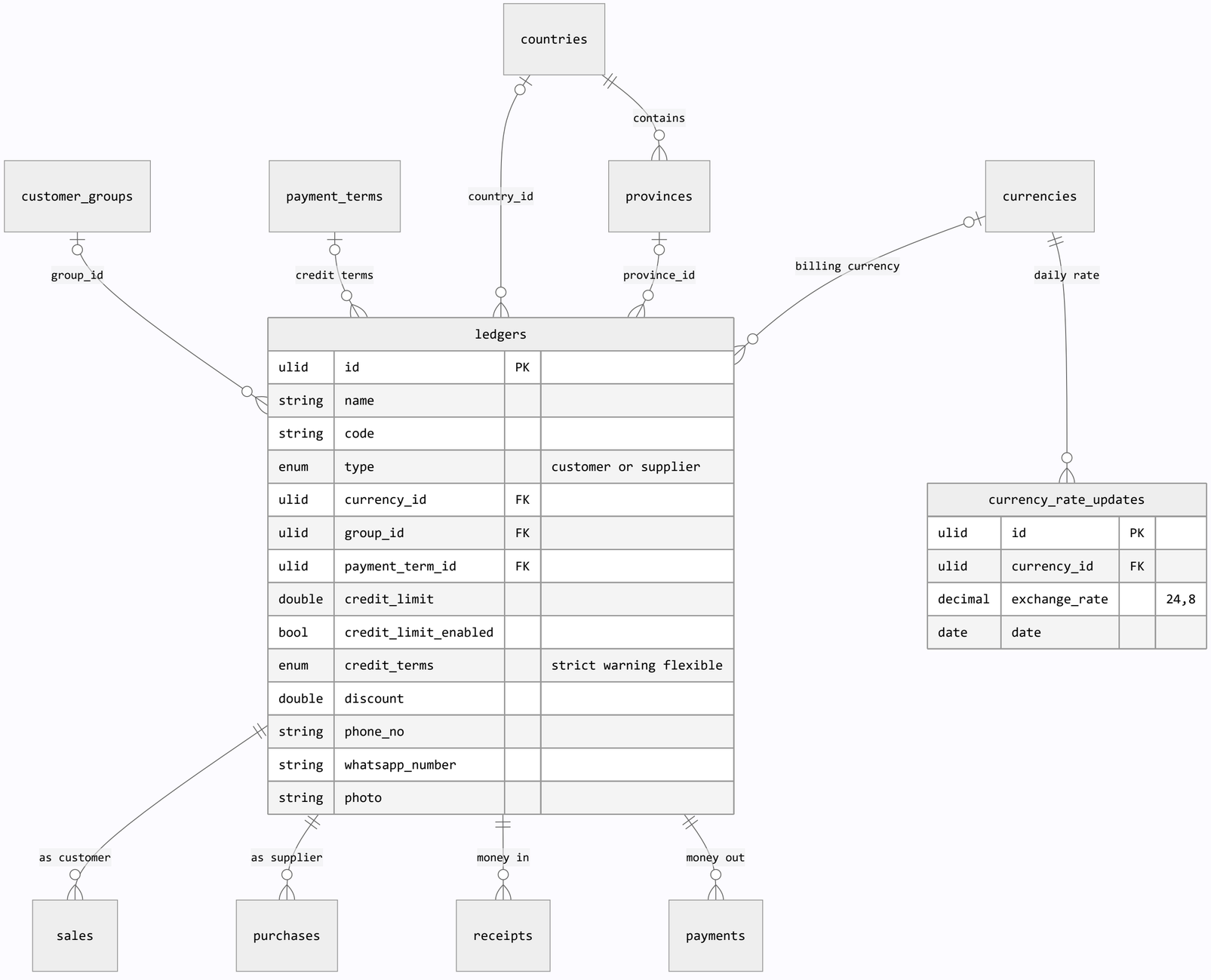
> **Thirteen document types, one posting table — and no constraint tying them together.** Reversals are written as further transactions carrying a `reversal` marker rather than by deleting lines, so in practice the ledger is append-only even though its rows are soft-deletable.

7 TABLES · ACCOUNTING → PARTIES

Parties: one table for customers and suppliers

There is no customers table and no suppliers table. `ledgers.type` is an enum with exactly two values, and both `sales.customer_id` and `purchases.supplier_id` point at the same table.

PLATE 2 — PARTIES AND REFERENCE DATA



A party's currency is its own. `ledgers.currency_id` is what drives per-currency statements, while `currencies.exchange_rate` holds the current rate and `currency_rate_updates` the dated history at `decimal(24,8)` precision.

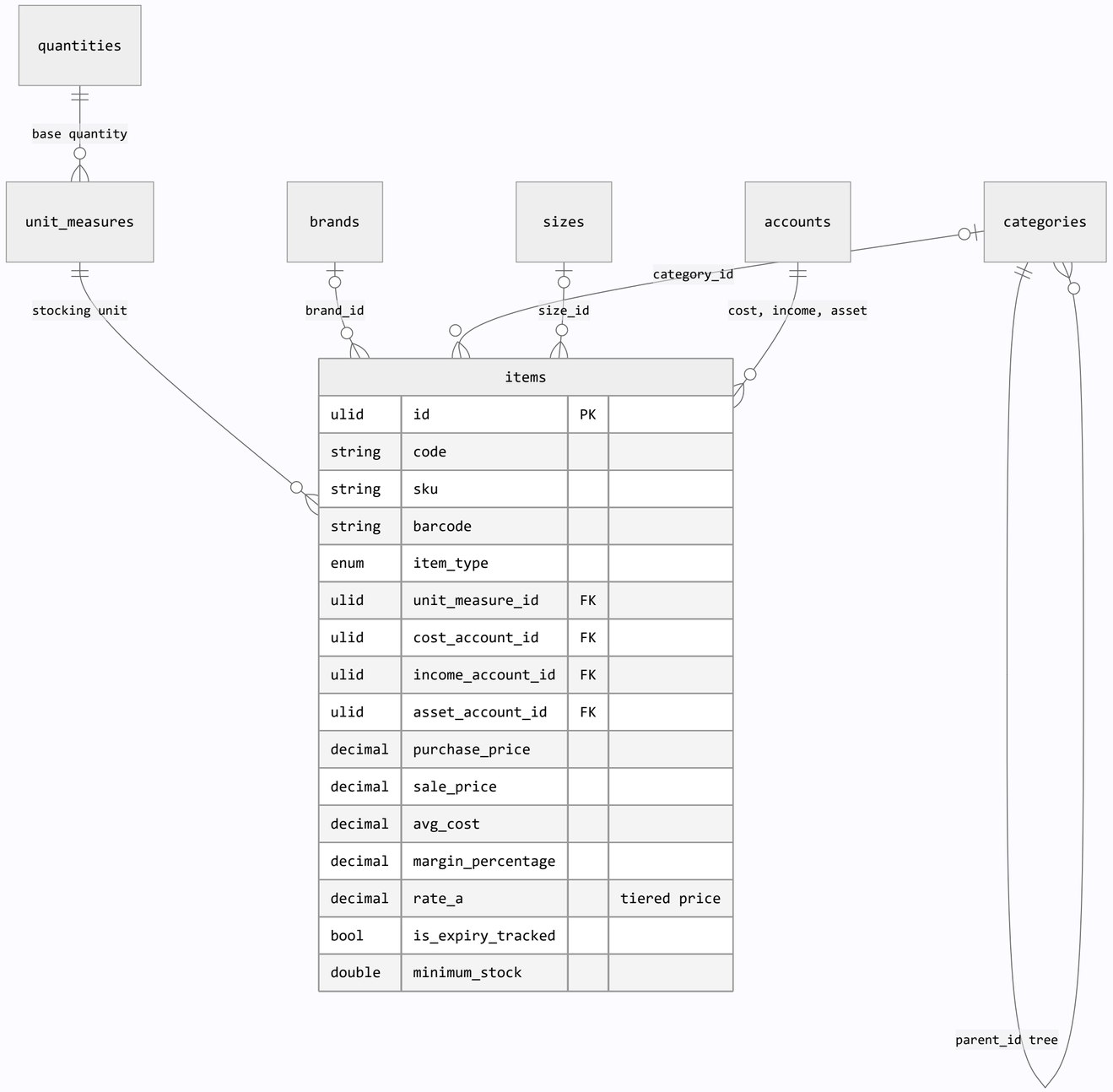
`countries · provinces`	The only tables with **no `branch_id` and no audit columns** — shared static reference data, seeded once, bilingual (`name_en` / `name_fa`).
`credit_limit_enabled`	The original `credit_limit_status` enum (`Block` / `Indicate`) was later renamed to `credit_terms` with `strict` / `warning` / `flexible`, plus a boolean switch.

14 TABLES · THE LARGEST DOMAIN

Inventory: master data, FIFO layers, and a cached balance

An item carries its own three GL accounts, so posting a stock document never has to guess where the money goes. Quantities are tracked twice on purpose: `stock_movements` is the immutable layer history, `stock_balances` the denormalised on-hand figure the UI reads.

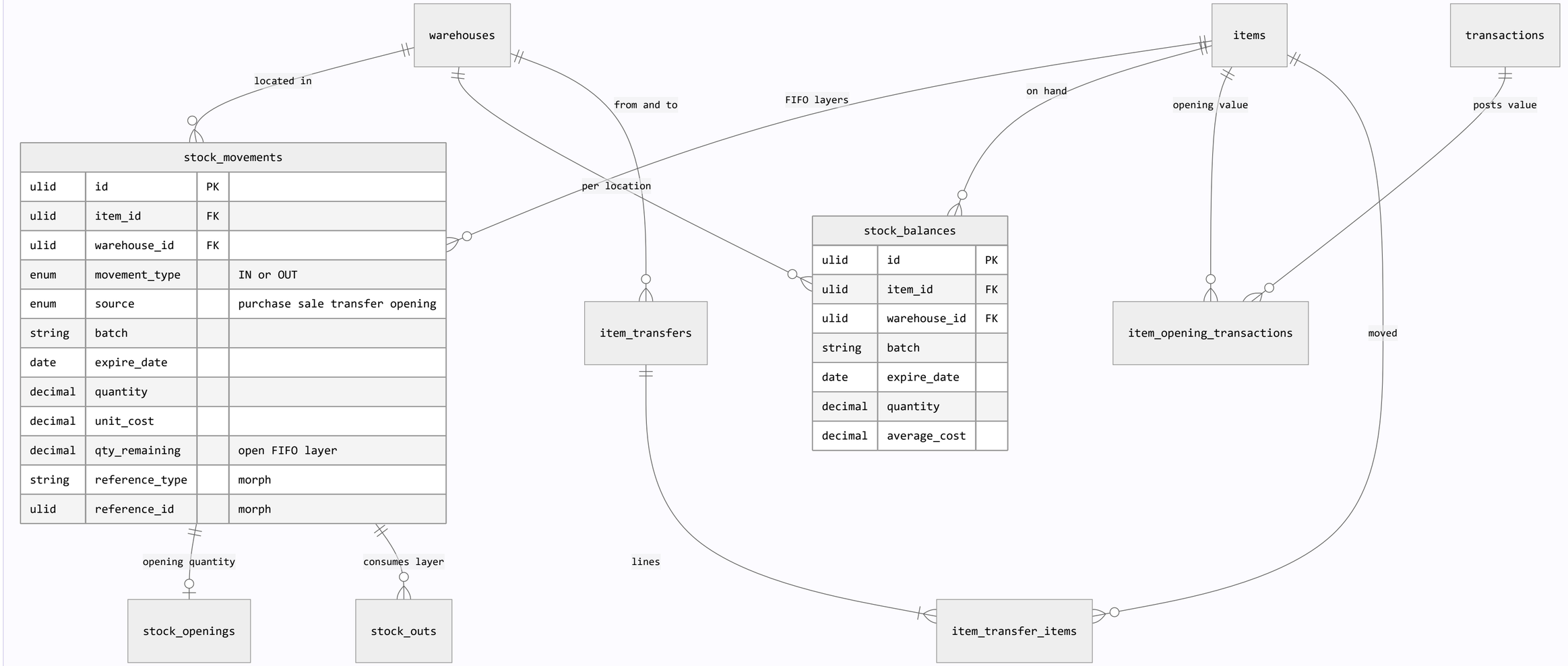
PLATE 3A — ITEM MASTER DATA



items			
ulid	id	PK	
string	code		
string	sku		
string	barcode		
enum	item_type		
ulid	unit_measure_id	FK	
ulid	cost_account_id	FK	
ulid	income_account_id	FK	
ulid	asset_account_id	FK	
decimal	purchase_price		
decimal	sale_price		
decimal	avg_cost		
decimal	margin_percentage		
decimal	rate_a		tiered price
bool	is_expiry_tracked		
double	minimum_stock		

An item cannot exist outside the chart of accounts. `cost_account_id`, `income_account_id` and `asset_account_id` are all non-nullable, which is what lets a stock document post itself without asking where the money goes.

PLATE 3B — STOCK LAYERS AND MOVEMENT



`qty_remaining` **is the whole FIFO mechanism.** An IN movement opens a layer at its `unit_cost`; an OUT consumes the oldest open layers and decrements their remainder, which is what makes cost of goods reproducible after the fact rather than a running average.

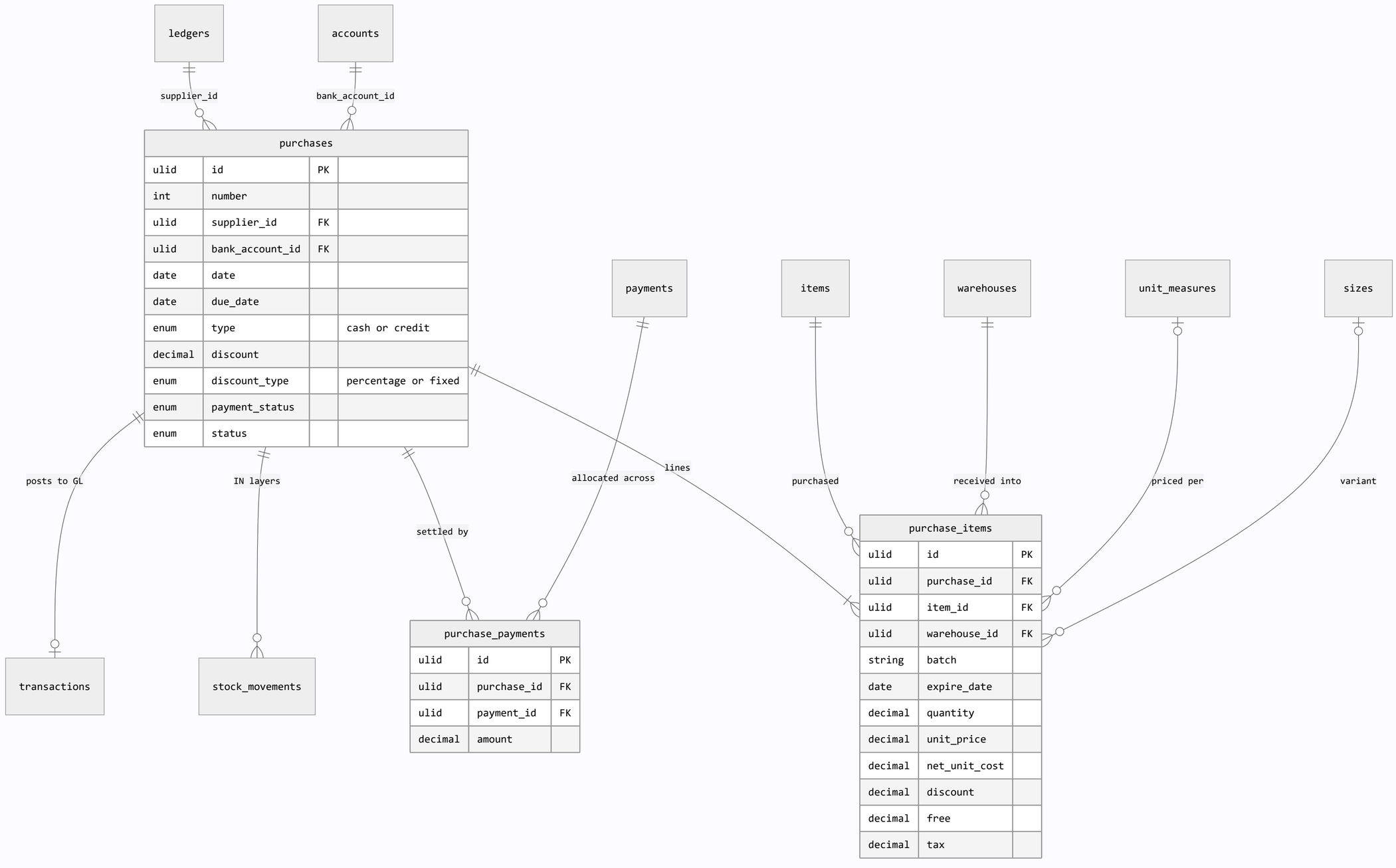
`items.*_account_id`	Three **non-nullable** FKs to `accounts` — cost, income and asset. An item cannot exist without a place in the chart of accounts.
`batch + expire_date`	Carried identically on `purchase_items`, `sale_items`, `item_transfer_items`, `stock_movements` and `stock_balances`. That repetition is what lets a batch be traced end to end without joining the document chain.
`stock_openings` vs `item_opening_transactions`	Two different openings, easy to confuse: the first links an item to its **opening quantity movement**, the second links it to the **transaction that values that quantity** in the GL.
`stock_outs`	The issue-side record, pointing back at the `stock_movement_id` layer it drew from, with its own polymorphic `source`.

3 TABLES · PLUS WHAT IT TOUCHES

Purchasing

A purchase names its supplier, its bank or cash account, and a payment type; its lines carry batch, expiry, warehouse and cost. Settlement is a many-to-many through `purchase_payments`, so one payment can clear several invoices.

PLATE 4 — PURCHASE TO PAYMENT



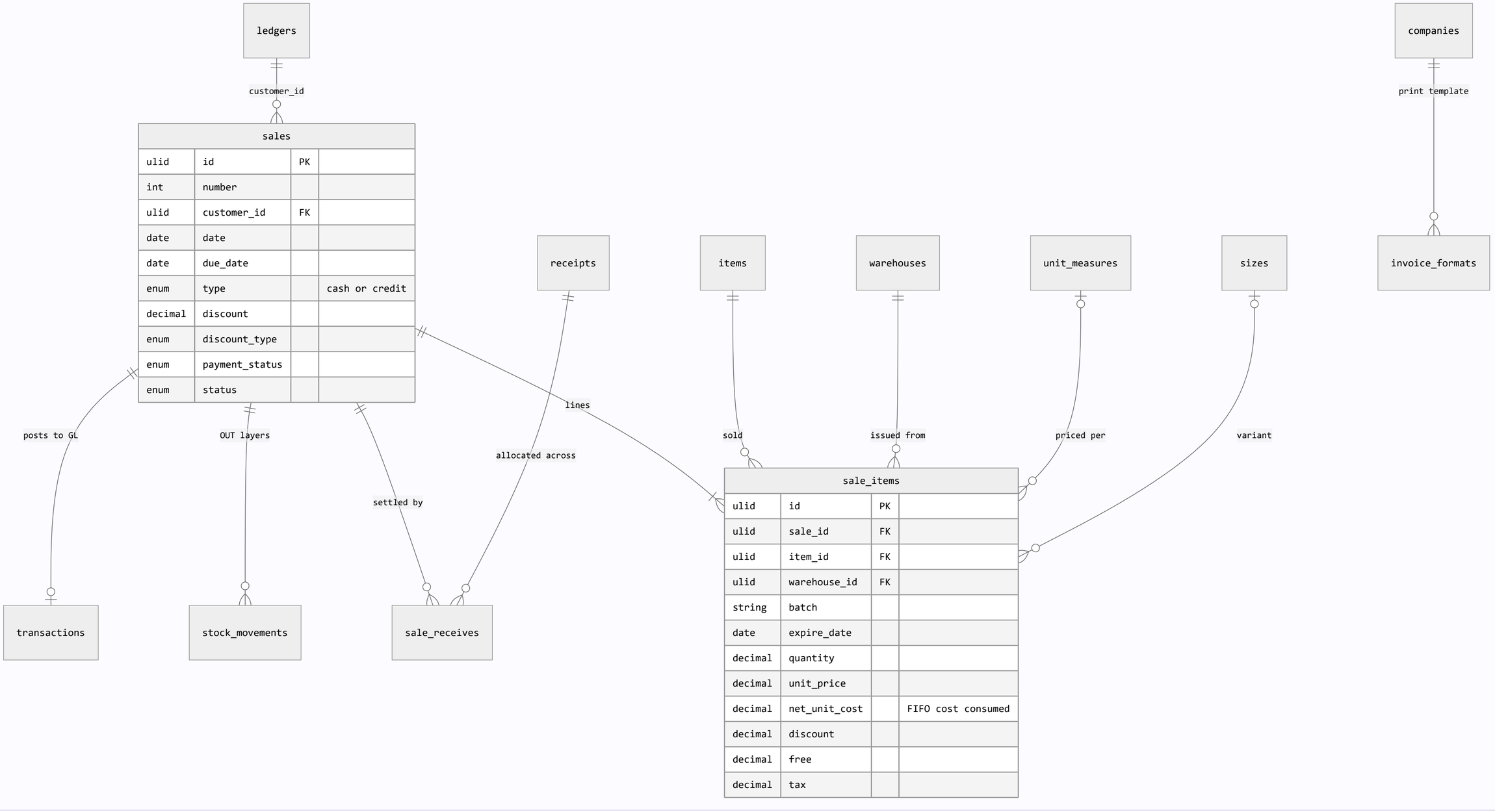
`net_unit_cost` is the number stock costing believes. `unit_price` is what the supplier invoiced; net unit cost is that price after discount, tax and free goods, and it is what becomes the FIFO layer's `unit_cost`.

4 TABLES · MIRRORS PURCHASING

Sales

Structurally the mirror of purchasing, with two differences: a sale has no bank account column — the receiving account comes from the receipt — and its printed output is driven by a per-company invoice format.

PLATE 5 — SALE TO RECEIPT



`sale_items.net_unit_cost` runs the other way. On a purchase it is a cost being created; on a sale it is the cost of the layers this line consumed — the figure the cost-of-goods posting is built from.

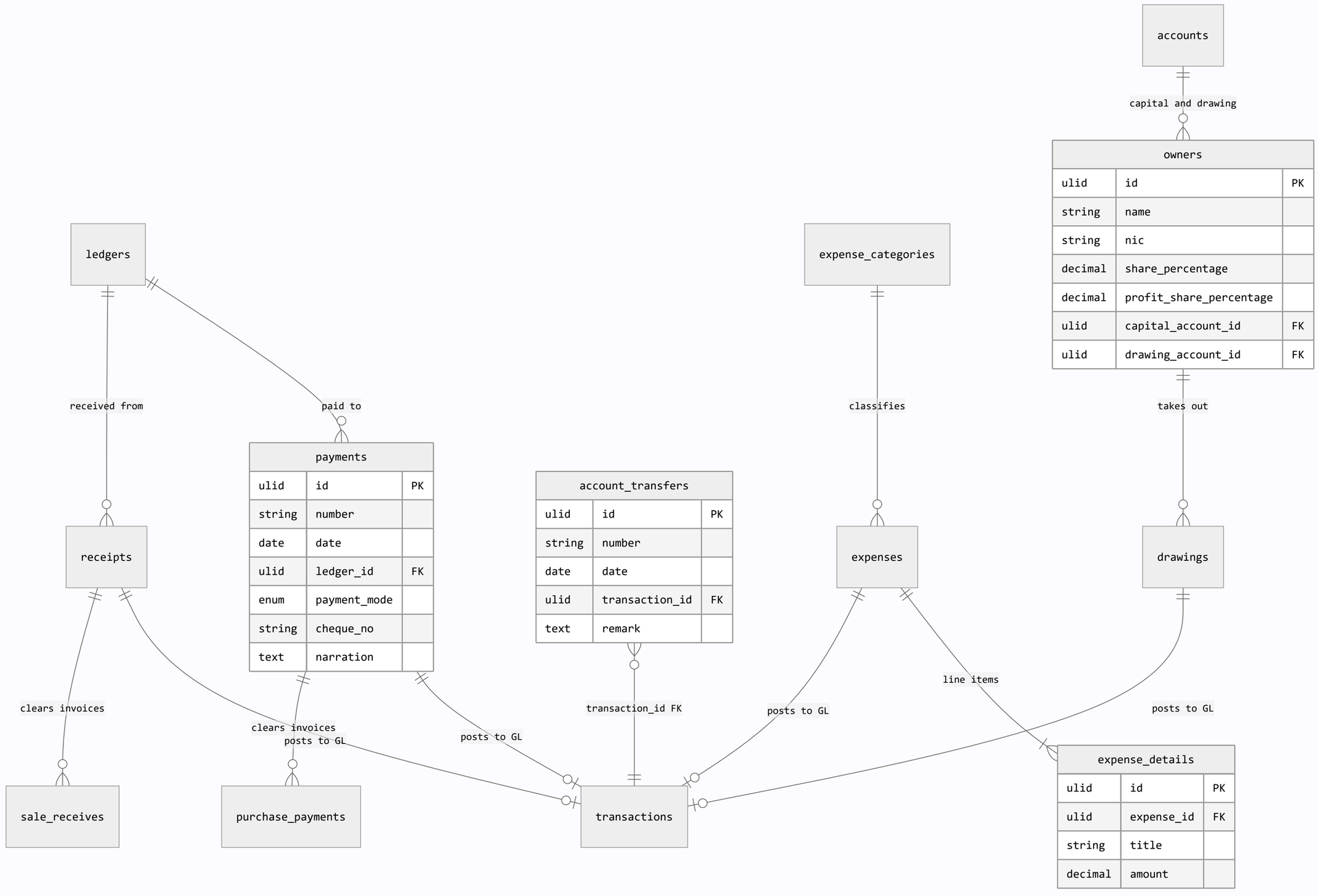
`invoice_formats`	The only other table besides `users` keyed to `company_id`, and the most JSON-heavy in the schema: `header_config`, `item_columns`, `optional_sections`, `appearance` and `margins` are all `json`, alongside `paper_size`, `language`, `direction` (`ltr` / `rtl`), `watermark_text` and `custom_css`. **Print layout is data, not code** — which is how one build serves Gregorian LTR and Jalali RTL invoices.

8 TABLES · TREASURY, EXPENSES, EQUITY

Cash, expenses and owner equity

Every table here is a thin document that exists to carry a date, a party and a narration; the amounts live in the transaction it posts. That is why `payments` and `receipts` have no amount column at all.

PLATE 6 — MONEY MOVEMENT



`account_transfers` is the one document that points the other way. Every other document is found by the transaction's polymorphic reference; a transfer instead holds a real `transaction_id` foreign key, because a transfer *is* its two lines and has nothing else to say.

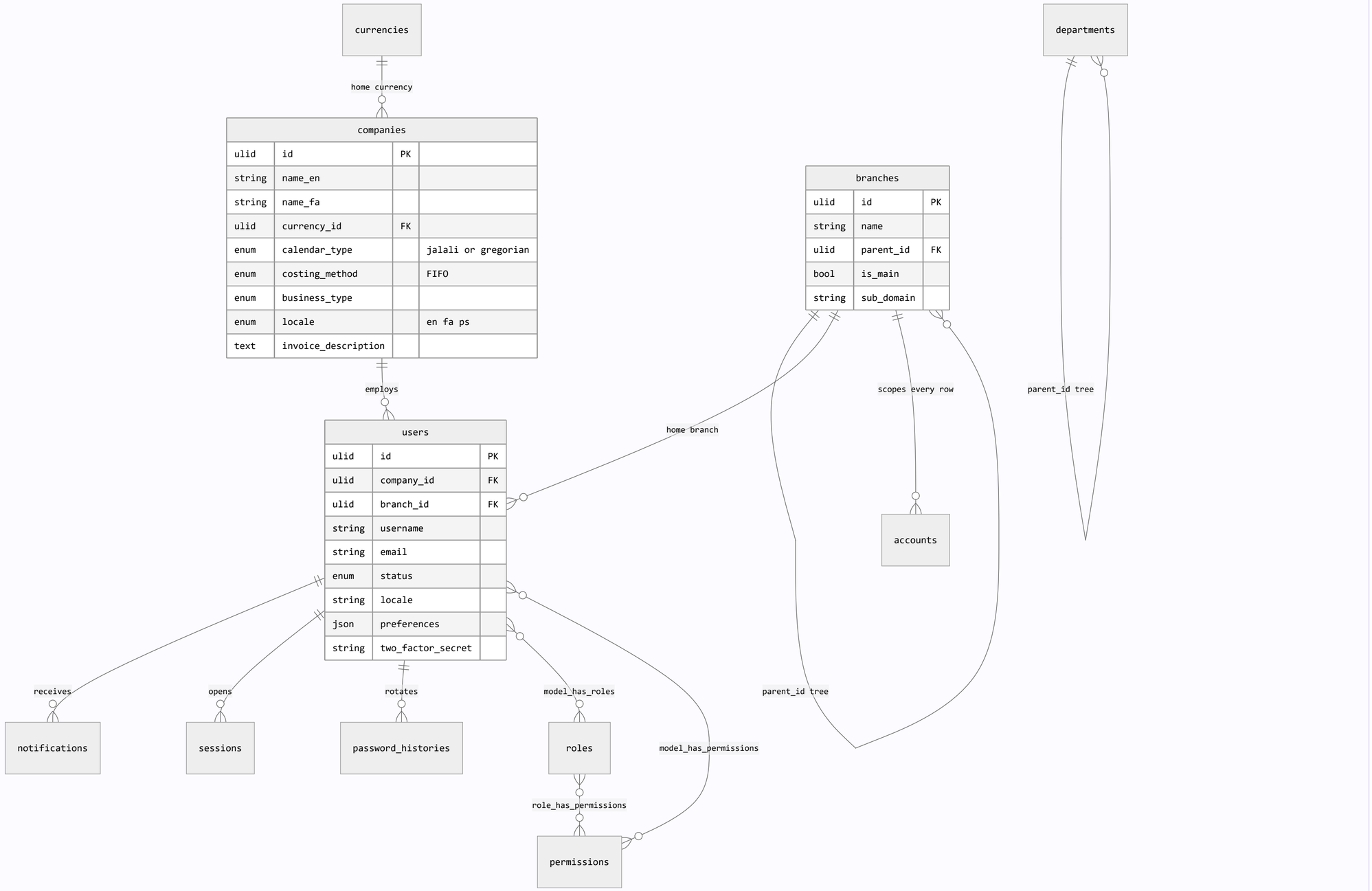
`bank account of a payment`	Not a column. `Payment::bankAccount()` resolves it by reading back the **credit line of its own transaction** — so the cash account a payment used is a derived fact, not a stored one.
`purchase_payments` · `sale_receives`	Allocation pivots with an `amount`, which is what makes partial settlement and the `payment_status` enum on the invoice possible.

11 TABLES · IDENTITY AND SCOPE

Tenancy, users and access

A user belongs to a company and sits in a branch; branches form a tree via `parent_id`.
Permissions are Spatie's five tables, re-keyed to ULIDs, with a `slug` added to roles.

PLATE 7 — WHO CAN ACT, AND WHERE



`branches ||--o{ accounts` stands in for 54 more edges. Every scoped table repeats the same `branch_id` FK; it is drawn once here rather than fifty-five times across the other plates.

> `HandleInertiaRequests` reads `user->company->calendar_type` on every request, and `users.company_id` is nullable. A user without a company therefore breaks the middleware before any controller runs — which is what `CheckCompany` exists to prevent.

3 TABLES · CROSS-CUTTING

System tables

These attach to everything, which makes them poor diagram subjects and good table rows. All three are deliberately outside the normal conventions.

TABLE	ATTACHES TO	WHY IT BREAKS THE PATTERN
`activity_logs`	`reference_type` + reference_id, module, user_id	Append-only audit trail: `timestampTz created_at` with no `updated_at`, no soft delete, and `jsonb` `old_values` / `new_values` / `metadata` for diffing.
`attachments`	`attachable_type` + attachable_id	Polymorphic file store — `disk`, `path`, `original_name`, `mime_type`, `size`. The row is the record; the bytes are on the disk.
`notifications`	`user_id`	One of the few tables with a real `cascadeOnDelete` FK, and no branch scope — a notification belongs to a person, not a branch.

FOUND WHILE DRAWING THIS

Loose ends worth a look

Six things in the schema that the diagrams above can't show, ordered by how likely they are to bite.

WHAT	WHERE	WHY IT MATTERS
`Missing deleted_at and branch_id`	`journal_entry_lines`	The create migration adds `deleted_by` and `timestamps()` but never `softDeletes()` or `branch_id`, and no later migration adds either — while **both** `JournalEntryLine` models declare `SoftDeletes` and one adds `BranchSpecific` / `HasBranch`. Those traits filter on columns that aren't there.
`Duplicate models, one table`	`JournalEntry, JournalEntryLine, PurchasePayment, SaleReceive`	Each exists at two namespaces (`App\Models\Accounting\` and `App\Models\JournalEntry\`; `App\Models\` and `App\Models\Purchase\` / `Sale\`) mapping to the same table with different traits. Two behaviours for one row depending on which class loaded it.
`Empty stub table`	`transaction_line_currencies`	Created with nothing but `id` and `timestamps`. Per-line foreign-currency amounts were planned here and never built; FX still lives on the transaction header rate.
`Models without tables`	`PosSession, AccountBalance`	Their migrations sit unapplied in `database/migrations/future tables/`, so `account_balance_snapshots`-style period caching and POS sessions are code-only today.
`Indexed but unconstrained`	`transaction_lines.ledger_id`	Indexed with no foreign key, unlike `transaction_id`, `account_id` and `journal_class_id` beside it. A deleted party can leave lines pointing nowhere.
`Two ledger code repairs`	`ledgers.code`	`prefix_numeric_ledger_codes` and `fix_duplicate_ledger_codes` both patch data in migrations. Codes are generated in application code, so the generator — not the data — is what needs to hold the invariant.

Drawn from `database/migrations` (89 files) and `app/Models` at commit `edbe7f6` on `main`. Relationships shown are those declared as foreign keys or Eloquent relations; polymorphic edges are labelled with the value stored in the type column.

Framework tables — `cache`, `cache_locks`, `jobs`, `job_batches`, `failed_jobs`, `sessions`, `password_reset_tokens`, `personal_access_tokens` — are excluded from the counts.